Database Design Analysis

Redundancy, anomalies, dependencies, and a normalized library database

Prepared by: Vigneshvar K K

Project: Library Management System. **Output convention:** result tables below are reproducible expected outputs from the fixed rows in the shared SQL script. They are not screenshots of MySQL execution.

1. Aim and scope

This practical examines how one flat borrowing register duplicates member, book, publisher, and loan facts. It identifies the resulting insertion, update, and deletion anomalies, lists the functional dependencies, and maps the data into nine relations in Third Normal Form (3NF).

The register stores one row per loan-author pair. A book with two authors therefore repeats the same copy, member, title, publisher, and loan details twice. The final model keeps the book and loan facts once and represents the many-to-many author relationship in a junction table.

2. Functional dependencies

Determinant	Dependent attributes	Reason
loan_id	copy_id, member_id, loan_date, due_date, return_date	A loan event identifies one copy, borrower, and its dates.
copy_id	book_id, accession_code, shelf_location, copy_status	Each physical copy has one catalog book and one location/status.
book_id	isbn, title, publication_year, publisher_id, category_id, list_price	A catalog record describes one edition/title entry.
publisher_id	publisher_name, city	Publisher details belong to the publisher, not to each book row.
category_id	category_name	A category ID identifies one category name.
member_id	member_no, full_name, email, phone, joined_on	Member details are independent of loan events.
author_id	author_name, country	Author details belong to an author.
(book_id, author_id)	author_role	A book-author pair has a role in that book.

In the flat register, the composite key is `(loan_id, author_id)`. The partial dependencies on `loan_id` and `author_id`, plus the transitive facts determined by `copy_id`, `book_id`, and `publisher_id`, show why one wide relation is a poor fit.

Normalization steps

Stage	Design change
1NF	Keep each field atomic. Authors are individual rows in book_authors, rather than a comma-separated list in one book field.
2NF	Move facts that depend on only loan_id or author_id out of the composite loan-author register.
3NF	Move publisher and category descriptions to their own relations; keep only their keys in books.

3. Runnable anomaly demonstration

Run this temporary-table example in MySQL Workbench. The complete block is also at the beginning of the shared SQL script. It inserts two author rows for each of two loans of the same book, then shows update and deletion anomalies.

```
CREATE TEMPORARY TABLE loan_register_raw (
    loan_id INT NOT NULL,
    author_id INT NOT NULL,
    copy_id INT NOT NULL,
    member_id INT NOT NULL,
    member_name VARCHAR(120) NOT NULL,
    member_email VARCHAR(160) NOT NULL,
    book_id INT NOT NULL,
    isbn CHAR(13) NOT NULL,
    title VARCHAR(180) NOT NULL,
    publisher_name VARCHAR(120) NOT NULL,
    author_name VARCHAR(120) NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE NULL,
    PRIMARY KEY (loan_id, author_id)
);

INSERT INTO loan_register_raw VALUES
    (1001, 1, 1, 1, 'Asha Mehta', 'asha.mehta@example.edu', 1,
     '9780133970777', 'Relational Database Basics', 'Meridian Academic',
     'Ada Sen', '2026-10-01', '2026-10-15', NULL),
    (1001, 2, 1, 1, 'Asha Mehta', 'asha.mehta@example.edu', 1,
     '9780133970777', 'Relational Database Basics', 'Meridian Academic',
     'Ravi Menon', '2026-10-01', '2026-10-15', NULL),
    (1002, 1, 2, 2, 'Kabir Shah', 'kabir.shah@example.edu', 1,
     '9780133970777', 'Relational Database Basics', 'Meridian Academic',
     'Ada Sen', '2026-10-10', '2026-10-24', NULL),
    (1002, 2, 2, 2, 'Kabir Shah', 'kabir.shah@example.edu', 1,
     '9780133970777', 'Relational Database Basics', 'Meridian Academic',
     'Ravi Menon', '2026-10-10', '2026-10-24', NULL);

SELECT loan_id, member_name, title, publisher_name, COUNT(*) AS author_rows
FROM loan_register_raw
GROUP BY loan_id, member_name, title, publisher_name
ORDER BY loan_id;

-- Deliberately update one duplicate row to make the update anomaly visible.
UPDATE loan_register_raw
SET publisher_name = 'Meridian Academic - Updated'
WHERE loan_id = 1001 AND author_id = 1;

SELECT DISTINCT publisher_name
FROM loan_register_raw
WHERE book_id = 1
ORDER BY publisher_name;

-- A member-only row cannot be represented because the key requires a loan and author.
-- Uncomment the next line to see the insertion anomaly as a NOT NULL/key error:
-- INSERT INTO loan_register_raw (member_id, member_name, member_email)
-- VALUES (6, 'New Member', 'new.member@example.edu');

DELETE FROM loan_register_raw WHERE loan_id = 1001;
SELECT COUNT(*) AS remaining_rows_for_asha
FROM loan_register_raw
WHERE member_id = 1;

DROP TEMPORARY TABLE loan_register_raw;
```

Initial duplicate summary

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

loan_id	member	book	publisher	authors
1001	Asha Mehta	Relational Database Basics	Meridian Academic	2
1002	Kabir Shah	Relational Database Basics	Meridian Academic	2

Update and deletion results

Query	Expected result
Distinct publisher names after changing only one duplicate row	Meridian Academic; Meridian Academic - Updated
Rows for member Asha after deleting her only loan rows	remaining_rows_for_asha = 0

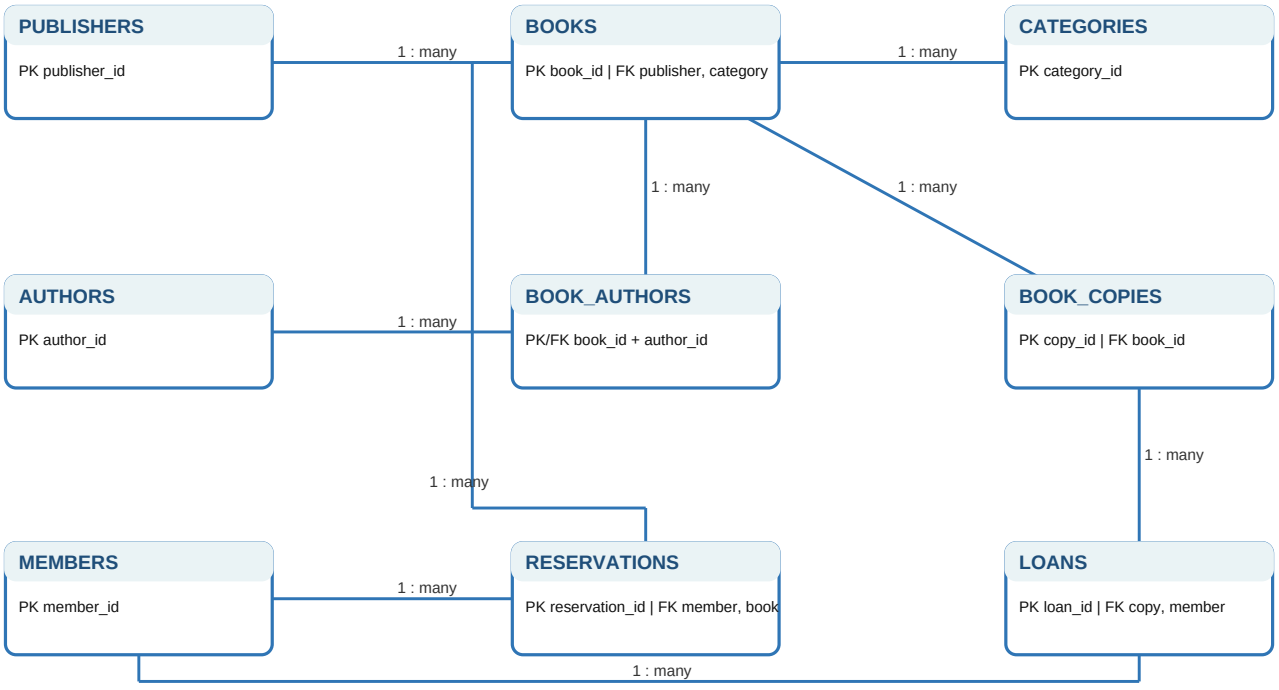
Insertion anomaly: the flat table has no independent member row. A member without a loan cannot be recorded without inventing a loan-author key. The shared SQL keeps the invalid insert commented so the main script can finish successfully; uncomment it only to observe the expected NOT NULL/key error.

4. Normalized relations

Relation	Primary key	Important references
publishers	publisher_id	None
categories	category_id	None
authors	author_id	None
books	book_id	publisher_id, category_id
book_authors	book_id + author_id	books, authors
book_copies	copy_id	books
members	member_id	None
loans	loan_id	book_copies, members
reservations	reservation_id	members, books

5. Entity relationship diagram

The diagram shows the nine normalized tables. The book-author junction resolves the many-to-many relationship, while each physical copy is the item named by a loan.



6. Normalized sample rows and output

After creating the schema with Assignment 5 and inserting the shared sample rows from Assignment 7, these joins retrieve the catalog record and its authors from separate normalized tables.

```
USE library_management_assignment;
SELECT b.book_id, b.title, p.publisher_name, c.category_name, b.list_price
FROM books AS b
JOIN publishers AS p ON p.publisher_id = b.publisher_id
JOIN categories AS c ON c.category_id = b.category_id
WHERE b.book_id = 1;

SELECT a.author_name, ba.author_role
FROM book_authors AS ba
JOIN authors AS a ON a.author_id = ba.author_id
WHERE ba.book_id = 1
ORDER BY a.author_name;
```

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

book_id	title	publisher_name	category_name	list_price
1	Relational Database Basics	Meridian Academic	Database	650.00

author_name	author_role
Ada Sen	Author
Ravi Menon	Author

The publisher, category, book, and author rows can be stored once and combined when a report needs them. That reduces repeated facts while preserving the relationships through keys.

Conclusion: The relations separate facts by their determinants, remove repeated publisher/member/title details from circulation rows, and preserve links with primary and foreign keys. The DDL and all deterministic sample rows appear in Assignments 5 and 7 and in the shared SQL file.